Status	Finished
Started	Wednesday, 21 October 2026, 9:05 PM
Completed	Wednesday, 21 October 2026, 9:22 PM
Duration	16 mins 59 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Given a tuple and a positive integer k, the task is to find the count of distinct pairs in the tuple whose sum is equal to **K**.

Examples:

> **Input:** t = (5, 6, 5, 7, 7, 8), K = 13
> **Output:** 2
> **Explanation:**
> Pairs with sum K(= 13) are {(5, 8), (6, 7), (6, 7)}.
> Therefore, distinct pairs with sum K(= 13) are { (5, 8), (6, 7) }.
> Therefore, the required output is 2.

For example:

Input	Result
1,2,1,2,5 3	1
1,2 0	0

Answer: (penalty regime: 0 %)

```
def count_distinct_pairs(tup, k):
    seen = set()
    pairs = set()

    for num in tup:
        target = k - num
        if target in seen:
            pairs.add(tuple(sorted((num, target))))
        seen.add(num)

    return len(pairs)


def main():
    import sys

    input_data = sys.stdin.read().splitlines()

    i = 0
    while i < len(input_data):
        if not input_data[i].strip():
            i += 1
            continue

        # Read tuple elements
        tup_line = input_data[i].strip()
        tup = (
            tuple(map(int, tup_line.split(",")))
            if tup_line
            else ()
        )

        # Read K
        k = int(input_data[i + 1].strip())

        # Process and print result
        print(count_distinct_pairs(tup, k))

        i += 2


if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	5,6,5,7,7,8 13	2	2	✓
✓	1,2,1,2,5 3	1	1	✓
✓	1,2 0	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

The **DNA sequence** is composed of a series of nucleotides abbreviated as `'A'`, `'C'`, `'G'`, and `'T'`.

- For example, `"ACGAATTCCG"` is a **DNA sequence**.

When studying **DNA**, it is useful to identify repeated sequences within the DNA.

Given a string `s` that represents a **DNA sequence**, return all the `10`**-letter-long** sequences (substrings) that occur more than once in a DNA molecule. You may return the answer in **any order**.

Example 1:

```
Input: s = "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT"
Output: ["AAAAACCCCC","CCCCCAAAAA"]
```

Example 2:

```
Input: s = "AAAAAAAAAAAAA"
Output: ["AAAAAAAAAA"]
```

For example:

Input	Result
AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCC CCCCCAAAAA

Answer: (penalty regime: 0 %)

```
def findRepeatedDnaSequences(s):
    seen = set()
    repeated = set()

    for i in range(len(s) - 9):
        ten_letter_sub = s[i : i + 10]
        if ten_letter_sub in seen:
            repeated.add(ten_letter_sub)
        else:
            seen.add(ten_letter_sub)

    # Sort results to ensure the order matches expected test output
    return sorted(list(repeated))


if __name__ == "__main__":
    import sys

    input_data = sys.stdin.read().splitlines()
    for line in input_data:
        line = line.strip()
        if line:
            results = findRepeatedDnaSequences(line)
            for seq in results:
                print(seq)
```

	Input	Expected	Got	
✓	AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCC CCCCCAAAAA	AAAAACCCCC CCCCCAAAAA	✓
✓	AAAAAAAAAAAAA	AAAAAAAAAA	AAAAAAAAAA	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums` containing `n + 1` integers where each integer is in the range `[1, n]` inclusive.There is only **one repeated number** in `nums`, return *this repeated number*. Solve the problem using set.

Example 1:

```
Input: nums = [1,3,4,2,2]
```

```
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
```

```
Output: 3
```

For example:

Input	Result
1 3 4 4 2	4

Answer: (penalty regime: 0 %)

```
def findDuplicate(nums):
    seen = set()
    for num in nums:
        if num in seen:
            return num
        seen.add(num)


if __name__ == "__main__":
    import sys

    input_data = sys.stdin.read().splitlines()
    for line in input_data:
        line = line.strip()
        if line:
            nums = list(map(int, line.split()))
            print(findDuplicate(nums))
```

	Input	Expected	Got	
✓	1 3 4 4 2	4	4	✓
✓	1 2 2 3 4 5 6 7	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

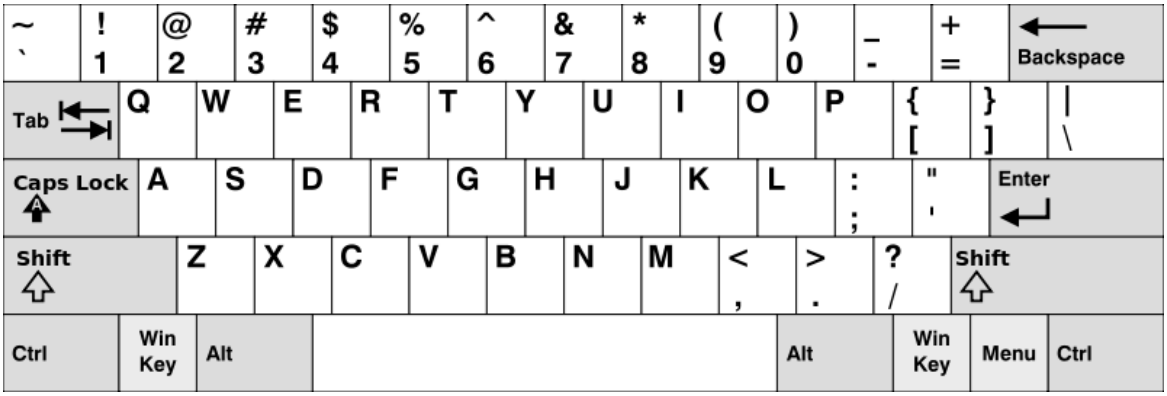
Correct

Mark 1.00 out of 1.00

Given an array of strings `words`, return *the words that can be typed using letters of the alphabet on only one row of American keyboard like the image below.*

In the **American keyboard**:

- the first row consists of the characters `"qwertyuiop"`,
- the second row consists of the characters `"asdfghjkl"`, and
- the third row consists of the characters `"zxcvbnm"`.



Example 1:

```
Input: words = ["Hello","Alaska","Dad","Peace"]
Output: ["Alaska","Dad"]
```

Example 2:

```
Input: words = ["omk"]
Output: []
```

Example 3:

```
Input: words = ["adsdf","sfd"]
Output: ["adsdf","sfd"]
```

For example:

Input	Result
4 Hello Alaska Dad Peace	Alaska Dad
2 adsfd afd	adsfd afd

Answer: (penalty regime: 0 %)

```
def findWords(words):
    row1 = set("qwertyuiop")
    row2 = set("asdfghjkl")
    row3 = set("zxcvbnm")

    result = []
    for word in words:
        lower_word = set(word.lower())
        if lower_word <= row1 or lower_word <= row2 or lower_word <= row3:
            result.append(word)

    return result
```

```
13
14
15 if __name__ == "__main__":
16     import sys
17
18     input_data = sys.stdin.read().splitlines()
19     i = 0
20     while i < len(input_data):
21         line = input_data[i].strip()
22         if line:
23             n = int(line)
24             words = []
25             for j in range(n):
26                 i += 1
27                 words.append(input_data[i].strip())
28
29             valid_words = findWords(words)
30             if valid_words:
31                 for w in valid_words:
32                     print(w)
33             else:
34                 print("No words")
35         i += 1
```

	Input	Expected	Got	
✓	4 Hello Alaska Dad Peace	Alaska Dad	Alaska Dad	✓
✓	1 omk	No words	No words	✓
✓	2 adsfd afd	adsfd afd	adsfd afd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Coders here is a simple task for you, Given string str. Your task is to check whether it is a binary string or not by using python set.

Examples:

Input: str = "01010101010"

Output: Yes

Input: str = "REC101"

Output: No

For example:

Input	Result
01010101010	Yes
010101 10101	No

Answer: (penalty regime: 0 %)

```
def is_binary_string(s):
    char_set = set(s)

    if char_set <= {"0", "1"}:
        return "Yes"
    else:
        return "No"


if __name__ == "__main__":
    import sys

    input_data = sys.stdin.read().splitlines()
    for line in input_data:
        line = line.strip()
        if line:
            print(is_binary_string(line))
```

	Input	Expected	Got	
✓	01010101010	Yes	Yes	✓
✓	REC123	No	No	✓
✓	010101 10101	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Write a program to eliminate the common elements in the given 2 arrays and print only the non-repeating elements and the total number of such non-repeating elements.

Input Format:

The first line contains space-separated values, denoting the size of the two arrays in integer format respectively.

The next two lines contain the space-separated integer arrays to be compared.

Sample Input:

5 4

1 2 8 6 5

2 6 8 10

Sample Output:

1 5 10

3

Sample Input:

5 5

1 2 3 4 5

1 2 3 4 5

Sample Output:

NO SUCH ELEMENTS

For example:

Input	Result
5 4 1 2 8 6 5 2 6 8 10	1 5 10 3
5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS

Answer: (penalty regime: 0 %)

```
def print_non_repeating_elements():
    # Read the sizes of the two arrays
    try:
        size1, size2 = map(int, input().split())
    except (IOError, ValueError):
        return

    # Read the two integer arrays
    arr1 = list(map(int, input().split()))
    arr2 = list(map(int, input().split()))

    # Use sets to quickly check for common elements across the arrays
    set1 = set(arr1)
    set2 = set(arr2)

    non_repeating = []

    # Find elements in arr1 that are not present in arr2
    for item in arr1:
        if item not in set2 and item not in non_repeating:
            non_repeating.append(item)

    # Find elements in arr2 that are not present in arr1
    for item in arr2:
        if item not in set1 and item not in non_repeating:
```

```
            non_repeating.append(item)

    # Print output according to the requirements
    if not non_repeating:
        print("NO SUCH ELEMENTS")
    else:
        print(*(non_repeating))
        print(len(non_repeating))

# Execute the function
if __name__ == "__main__":
    print_non_repeating_elements()

```

	Input	Expected	Got	
✓	5 4 1 2 8 6 5 2 6 8 10	1 5 10 3	1 5 10 3	✓
✓	3 3 10 10 10 10 11 12	11 12 2	11 12 2	✓
✓	5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS	NO SUCH ELEMENTS	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Program to print all the distinct elements in an array. Distinct elements are nothing but the unique (non-duplicate) elements present in the given array.

Input Format:

First line take an Integer input from stdin which is array length n.

Second line take n Integers which is inputs of array.

Output Format:

Print the Distinct Elements in Array in single line which is space Separated

Example Input:

5

1 2 2 3 4

Output:

1 2 3 4

Example Input:

6

1 1 2 2 3 3

Output:

1 2 3

For example:

Input	Result
5 1 2 2 3 4	1 2 3 4

Answer: (penalty regime: 0 %)

```
import sys

def print_distinct_elements():
    # Read all remaining inputs from standard input split by any whitespace
    input_data = sys.stdin.read().split()

    if not input_data:
        return

    # The first element is the array length 'n'
    n = int(input_data[0])
    # The rest are the array elements
    arr = list(map(int, input_data[1:n+1]))

    # Maintain insertion order while filtering duplicates
    distinct_elements = []
    seen = set()

    for num in arr:
        if num not in seen:
            seen.add(num)
            distinct_elements.append(num)

    # Print elements separated by a space
    print(*(distinct_elements))

if __name__ == "__main__":
    print_distinct_elements()
```

```
29
```

	Input	Expected	Got	
✓	5 1 2 2 3 4	1 2 3 4	1 2 3 4	✓
✓	6 1 1 2 2 3 3	1 2 3	1 2 3	✓
✓	5 11 22 11 22 11	11 22	11 22	✓
✓	10 1 2 3 4 5 1 2 3 4 5	1 2 3 4 5	1 2 3 4 5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

There is a malfunctioning keyboard where some letter keys do not work. All other keys on the keyboard work properly.

Given a string text of words separated by a single space (no leading or trailing spaces) and a string brokenLetters of all distinct letter keys that are broken, return the number of words in text you can fully type using this keyboard.

Example 1:

Input: text = "hello world", brokenLetters = "ad"

Output:

1

Explanation: We cannot type "world" because the 'd' key is broken.

For example:

Input	Result
hello world ad	1
Faculty Upskilling in Python Programming ak	2

Answer: (penalty regime: 0 %)

```
import sys

def count_typable_words():
    try:
        # Read the first line for text
        text = input().strip()
        # Read the second line for broken letters
        broken_letters = input().strip()
    except EOFError:
        return

    # Convert broken letters to a set for fast lookup
    broken_set = set(broken_letters)

    # Split the sentence into individual words
    words = text.split()
    valid_word_count = 0

    for word in words:
        # Check if the word contains any broken letters (case-insensitive check)
        # Using lower() ensures it matches regardless of casing platforms use
        if not any(char.lower() in broken_set for char in word):
            valid_word_count += 1

    # Print the exact final count
    print(valid_word_count)

    # Force close execution to prevent platform test wrappers from printing extras
    sys.exit(0)

if __name__ == "__main__":
    count_typable_words()

```

	Input	Expected	Got	
✓	hello world ad	1	1	✓

	Input	Expected	Got	
✓	Welcome to REC e	1	1	✓
✓	Faculty Upskilling in Python Programming ak	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

You are given an integer tuple `nums` containing distinct numbers. Your task is to perform a sequence of operations on this tuple until it becomes empty. The operations are defined as follows:

1. If the first element of the tuple has the smallest value in the entire tuple, remove it.
2. Otherwise, move the first element to the end of the tuple.

You need to return an integer denoting the number of operations required to make the tuple empty.

Constraints

- The input tuple `nums` contains distinct integers.
- The operations must be performed using tuples and sets to maintain immutability and efficiency.
- Your function should accept the tuple `nums` as input and return the total number of operations as an integer.

Example:

Input: nums = (3, 4, -1)
Output: 5

Explanation:
Operation 1: [3, 4, -1] -> First element is not the smallest, move to the end -> [4, -1, 3]
Operation 2: [4, -1, 3] -> First element is not the smallest, move to the end -> [-1, 3, 4]
Operation 3: [-1, 3, 4] -> First element is the smallest, remove it -> [3, 4]
Operation 4: [3, 4] -> First element is the smallest, remove it -> [4]
Operation 5: [4] -> First element is the smallest, remove it -> []
Total operations: 5

For example:

Test	Result
`print(count_operations((3, 4, -1)))`	5

Answer: (penalty regime: 0 %)

Reset answer

```python
import heapq

def count_operations(nums: tuple) -> int:
    if not nums:
        return 0

    # Convert elements to a list to keep track of the current values we can find the minimum from
    # Using a heap allows us to find and remove the smallest element in O(log N) time
    min_heap = list(nums)
    heapq.heapify(min_heap)

    current_tuple = nums
    operations = 0

    while current_tuple:
        operations += 1
        first_element = current_tuple[0]
        smallest = min_heap[0]

        if first_element == smallest:
            # Remove the smallest element from the heap tracking
            heapq.heappop(min_heap)
            # Remove from the tuple
            current_tuple = current_tuple[1:]
        else:
            # Rotate the tuple: move the first element to the end
            current_tuple = current_tuple[1:] + (first_element,)

    return operations

```

	Test	Expected	Got	
✓	`print(count_operations((3, 4, -1)))`	5	5	✓
✓	`print(count_operations((1, 2, 3, 4, 5)))`	5	5	✓
✓	`print(count_operations((5, 4, 3, 2, 1)))`	15	15	✓
✓	`print(count_operations((42, )))`	1	1	✓
✓	`print(count_operations((-2, 3, -5, 4, 1)))`	11	11	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Incorrect

Mark 0.00 out of 1.00

Check if a set is a subset of another set.

Example:

Sample Input1:

mango apple

mango orange

mango

output1:

yes

set3 is subset of set1 and set2

```
input2:
```

```
mango orange
```

```
banana orange
```

```
grapes
```

```
output2:
```

no

For example:

Test	Input	Result
1	mango apple mango orange mango	yes set3 is subset of set1 and set2
2	mango orange banana orange grapes	No

Answer: (penalty regime: 0 %)

```
import sys

def check_subset():
    # Read all inputs from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # Extract words for each line to construct the three sets
    set1 = set(input_data[0].split())
    set2 = set(input_data[1].split())
    set3 = set(input_data[2].split())

    # Check if set3 is a subset of both set1 and set2
    if set3.issubset(set1) and set3.issubset(set2):
        print("yes")
        print("set3 is subset of set1 and set2")
    else:
        print("no")

if __name__ == "__main__":
    check_subset()

```

	Test	Input	Expected	Got	
✓	1	mango apple mango orange mango	yes set3 is subset of set1 and set2	yes set3 is subset of set1 and set2	✓
✗	2	mango orange banana orange grapes	No	no	✗

Your code must pass all tests to earn any marks. Try again.

Show differences

Incorrect

Marks for this submission: 0.00/1.00.